

Dual Reward Structure

The reward has two components operating at different timescales. The local shaping reward is computed immediately after each element action. The global retrospective reward is computed once per remesh interval, after the solver advances the PDE. Neither Foucart nor DynAMO combines both levels of feedback — this dual structure is novel to this architecture.

Why Two Rewards?

The local reward teaches the agent which individual actions are appropriate: refine elements with high error, coarsen elements with low error, leave neutral elements alone. But locally correct decisions can produce a globally poor mesh — for example, the agent might correctly refine every high-error element it sees, exhaust the budget, and leave other regions completely unresolved.

The global reward assesses the *overall mesh quality* after the solver actually uses it to advance the PDE. It captures consequences that no per-element signal can: did the adapted mesh maintain accuracy everywhere during the solver advance? Were resources allocated effectively across the domain?

The two signals complement each other. Local shaping provides dense, immediate feedback on every decision, making learning tractable. Global retrospective provides the ground truth on what actually matters — mesh quality during PDE integration — but it’s sparse (one signal per ~ 45 element visits) and requires the RL algorithm’s credit assignment to propagate backward through individual decisions.

Classification Thresholds (Restated)

Both reward components classify elements using the same thresholds, computed once at the start of each remesh interval and held fixed:

$$e_{\max} = \alpha \cdot \|e\|_{\infty} \quad (\text{refinement threshold}) \quad (1)$$

$$e_{\min} = e_{\max}^{\beta} \quad (\text{coarsening threshold, with } \beta = 1.2) \quad (2)$$

Elements with $e_k > e_{\max}$ are *under-refined*. Elements with $e_k < e_{\min}$ are *over-refined*. Elements in between are in the *neutral zone*. The gap between $e_{\max}$ and $e_{\min}$ (controlled by β) prevents oscillation — an element coarsened out of the neutral zone doesn’t immediately land above the refinement threshold.

Local Shaping Reward

Computed immediately after each element action, before any solver step. Classifies the agent’s action against the element’s current error indicator e_k (measured before the action is executed):

Error Region	Refine	Do-Nothing	Coarsen
$e_k > e_{\max}$ (under-refined)	0 (correct)	0 (acceptable)	$-p_{ur} \cdot \log_{10} \frac{e_k}{e_{\max}} $ (wrong)
$e_{\min} \leq e_k \leq e_{\max}$ (neutral)	0	0	0
$e_k < e_{\min}$ (over-refined)	$-p_{or} \cdot \log_{10} \frac{e_k}{e_{\min}} $ (wrong)	0 (acceptable)	$+p_{cr} \cdot \log_{10} \frac{e_k}{e_{\min}} $ (correct)

Key design choices in this table:

1. **Do-nothing is never penalized.** The agent may have strategic reasons to defer action — perhaps the budget is nearly exhausted and a higher-priority element is coming later in the queue. The global retrospective handles cases where inaction leads to a bad mesh.
2. **Positive coarsening reward (p_{cr}).** In a budget-constrained system, correct coarsening frees resources for better allocation elsewhere. Without a positive signal, the agent has no immediate incentive to coarsen — it would only learn to coarsen indirectly through the global reward, which is much slower. The reward magnitude ($p_{cr} = 2.0$) is deliberately smaller than the penalty weights ($p_{ur} = 10, p_{or} = 5$) so that gaming coarsening for reward is not attractive.
3. **Logarithmic penalty scaling.** An element with error $10\times$ above $e_{\max}$ incurs $p_{ur} \times 1$. An element $100\times$ above incurs $p_{ur} \times 2$. This provides smooth gradients without extreme penalties, and ensures the reward is scale-invariant.
4. **Penalties use the pre-action error.** The error indicator e_k is captured before the action executes, since the element may no longer exist after refinement or coarsening. The reward evaluates the decision against the information the agent had when deciding.

Parameter values: $p_{ur} = 10$ (under-refinement penalty), $p_{or} = 5$ (over-refinement penalty), $p_{cr} = 2.0$ (correct coarsening reward). The 2:1 asymmetry between p_{ur} and p_{or} reflects that under-refinement introduces irreversible discretization error, while over-refinement only wastes compute.

Global Retrospective Reward

Computed once per remesh interval, after all adaptation rounds complete and the solver advances the PDE by time T . Assesses the adapted mesh quality using **max-over-interval errors**:

$$\hat{e}_k = \max_{t \in [t_\tau, t_\tau + T]} e_k(t) \quad (3)$$

The maximum error each element experiences across all CFL sub-steps during the solver advance. This is accumulated inside the solver’s time-stepping loop.

Why max-over-interval instead of instantaneous error? Consider a wave pulse advecting through an element during $[t_\tau, t_\tau + T]$. If the interval is long enough, the pulse enters and exits the element entirely within one interval. The instantaneous error at $t_\tau + T$ would be near-zero because the pulse has moved on. An agent that correctly anticipated and pre-refined this element would be *penalized for over-refinement* under an instantaneous error reward. The max-over-interval captures that the element *did* experience high error during the interval, rewarding the correct anticipatory decision.

The global reward is the negative sum of penalties across all active elements:

$$r_{\text{global}} = - \sum_{k \in \mathcal{A}} \text{penalty}_k \quad (4)$$

where:

$$\text{penalty}_k = \begin{cases} p_{ur} \cdot \left| \log_{10} \left(\frac{\hat{e}_k}{e_{\max}} \right) \right| & \text{if } \hat{e}_k > e_{\max} \text{ and } \ell_k < \ell_{\max} \\ p_{or} \cdot \left| \log_{10} \left(\frac{\hat{e}_k}{e_{\min}} \right) \right| & \text{if } \hat{e}_k < e_{\min} \text{ and } \ell_k > 0 \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

The conditional guards are important:

- **Under-refinement penalty requires $\ell_k < \ell_{\max}$.** An element at maximum refinement level with high error is not the agent’s fault — there was no further refinement available.
- **Over-refinement penalty requires $\ell_k > 0$.** A base-level element with low error simply has a well-resolved region — there is nothing to coarsen.

The thresholds $e_{\max}$ and $e_{\min}$ are the same ones computed at the start of the remesh interval (before adaptation). This ensures the global reward evaluates decisions against the error landscape the agent observed when deciding, not the post-advance error landscape it couldn’t have anticipated.

Reward Delivery to the RL Algorithm

The RL algorithm (MaskablePPO via SB3) expects a single scalar reward per `step()` call. The two reward components are combined as follows:

On **most steps** (element visits within a round):

$$\text{reward} = \lambda \cdot r_{\text{local}}$$

On the **interval-terminal step** (last element of the last round in a remesh interval):

$$\text{reward} = \lambda \cdot r_{\text{local}} + r_{\text{global}}$$

where $\lambda = 0.1$ is the local-to-global weighting factor. The small λ reflects that local shaping is a learning aid — the global retrospective is the true measure of mesh quality.

The RL algorithm’s credit assignment mechanism (Generalized Advantage Estimation, or GAE) propagates the global reward backward through all preceding element visits in the interval. Over many training episodes, each element visit learns how its action contributed to the overall mesh quality assessed by the global reward. This is how the agent develops strategic behavior — such as conserving budget for high-priority regions — that no per-element signal could teach directly.

Key Points

1. **Local shaping provides dense, immediate feedback** on individual decisions. Global retrospective provides sparse, delayed ground truth on overall mesh quality. Both are needed.
2. **Do-nothing is never penalized locally.** Strategic inaction is a valid choice that the global reward will evaluate.
3. **Max-over-interval errors prevent rewarding anticipatory refinement as over-refinement.** Transient high-error events are captured even if the feature has moved on by the end of the interval.
4. **Conditional guards** prevent penalizing the agent for situations it cannot control (max-level elements with high error, base-level elements with low error).
5. **The same thresholds** ($e_{\max}$, $e_{\min}$) are used for both local and global rewards, ensuring consistency: the agent is judged by the same criteria it observed when deciding.